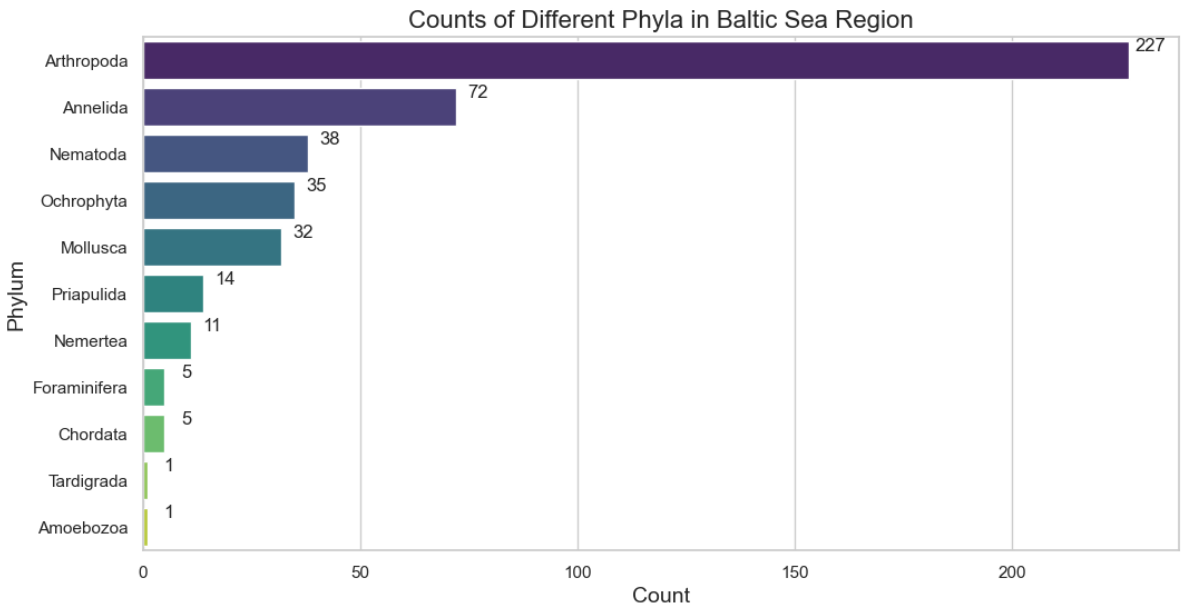
In [243…

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('Baltic_revised.csv')
#species_counts = df['scientificname'].value_counts()

# Print the species and their counts
#for species, count in species_counts.items():
#    print(f"Species: {species}, Count: {count}")
```

In [251…

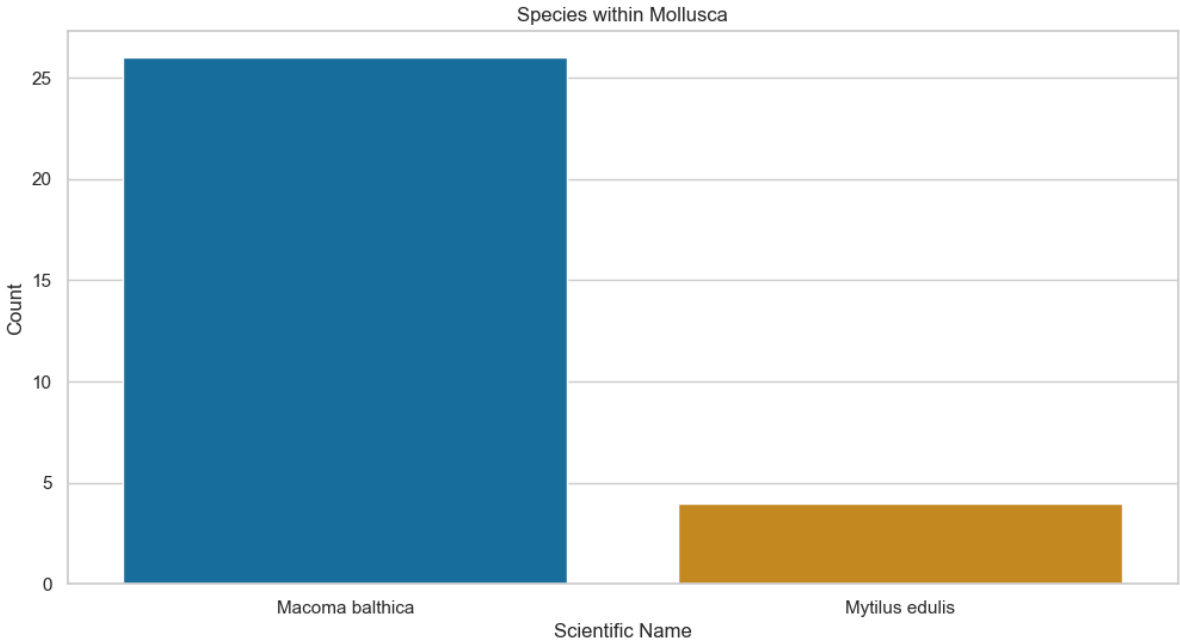
```
# Group the data by 'phylum' and count the occurrences of each phylum
phylum_counts = df['phylum'].value_counts().reset_index()
phylum_counts.columns = ['Phylum', 'Count']


plt.figure(figsize=(12, 6))
sns.set(style="whitegrid")
ax = sns.barplot(x="Count", y="Phylum", data=phylum_counts, palette="viridis


plt.xlabel('Count', fontsize=14)
plt.ylabel('Phylum', fontsize=14)
plt.title('Counts of Different Phyla in Baltic Sea Region', fontsize=16)


for p in ax.patches:
    width = p.get_width()
    plt.text(width + 5, p.get_y() + p.get_height() / 2 - 0.2, int(width), ha

plt.savefig("phylum_counts_plot_Baltic_sea.pdf", format="pdf")
plt.show()
```



In [245…

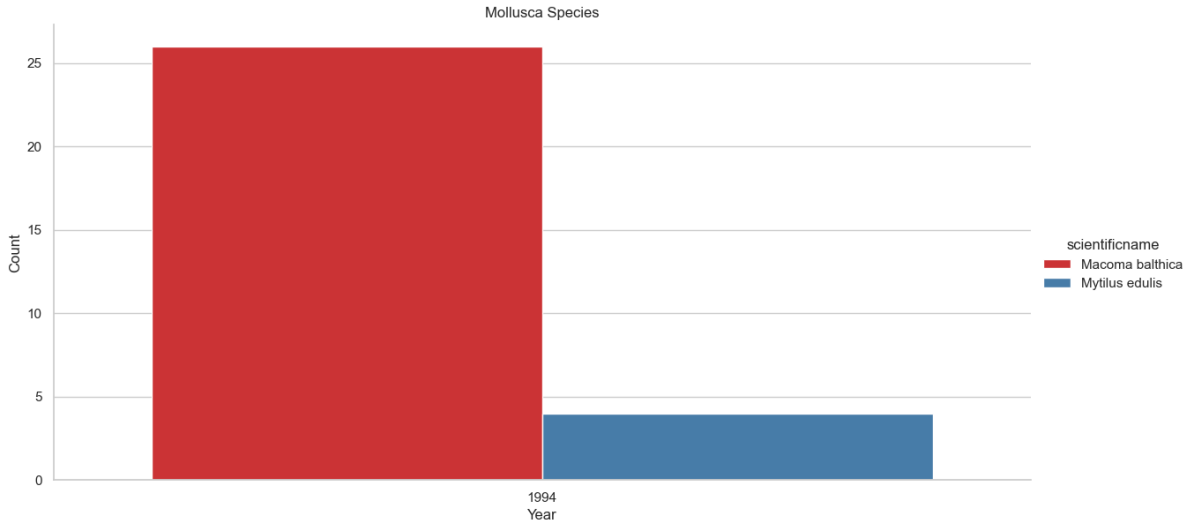
```
#MOllusca count

# Filter the DataFrame for rows where 'Phylum' is 'Mollusca'
mollusca_data = df[df['phylum'] == 'Mollusca']

# Count the occurrences of each unique scientific name within the 'Mollusca
scientificname_counts = mollusca_data['scientificname'].value_counts().reset
scientificname_counts.columns = ['Scientific Name', 'Count']
```

```
# Filter for scientific names with a count greater than or equal to 3
scientificname_counts = scientificname_counts[scientificname_counts['Count']


scientificname_counts = scientificname_counts.sort_values(by='Count', ascenc

plt.figure(figsize=(12, 6))
sns.barplot(x='Scientific Name', y='Count', data=scientificname_counts, pale
plt.xlabel('Scientific Name')
plt.ylabel('Count')
plt.title('Species within Mollusca')
plt.show()
```



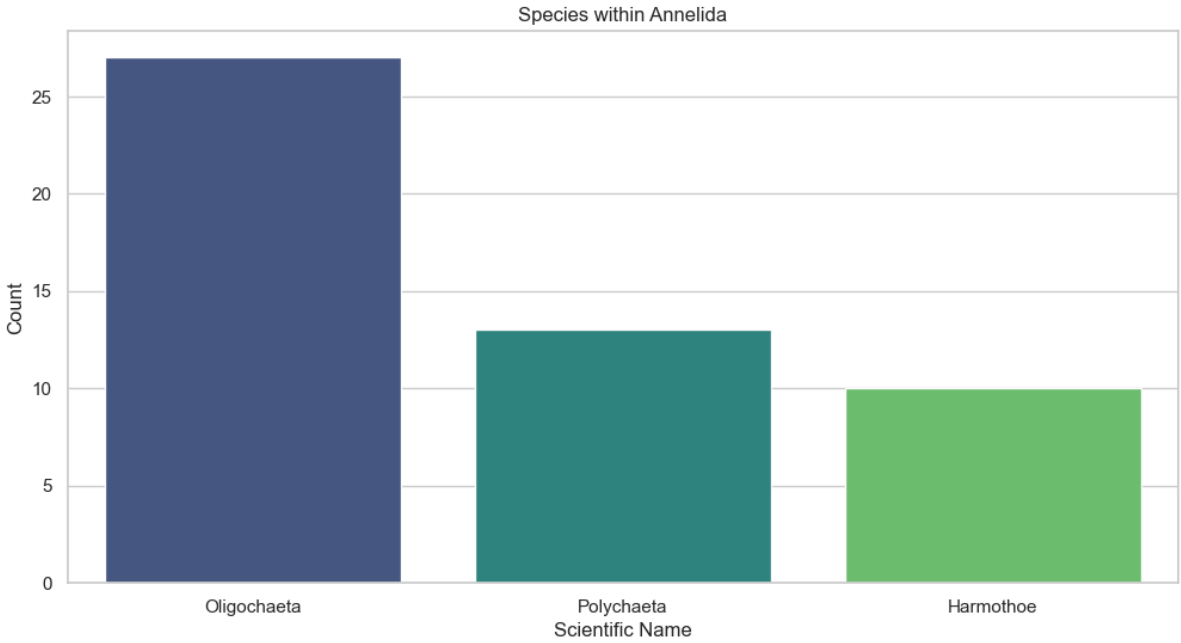
In [246…

```
#mollusca count over years


years_to_analyze = [1994, 1995]


species_to_plot = ['Macoma balthica', 'Mytilus edulis']

# Filter the DataFrame for the selected species and years
filtered_data = df[df['yearcollected'].isin(years_to_analyze) & df['scientif


g = sns.catplot(data=filtered_data, x='yearcollected', kind='count', hue='sc
g.set_axis_labels('Year', 'Count')
g.set_titles('Species: {col_name}')
plt.title('Mollusca Species')
plt.show()
```



In [247…

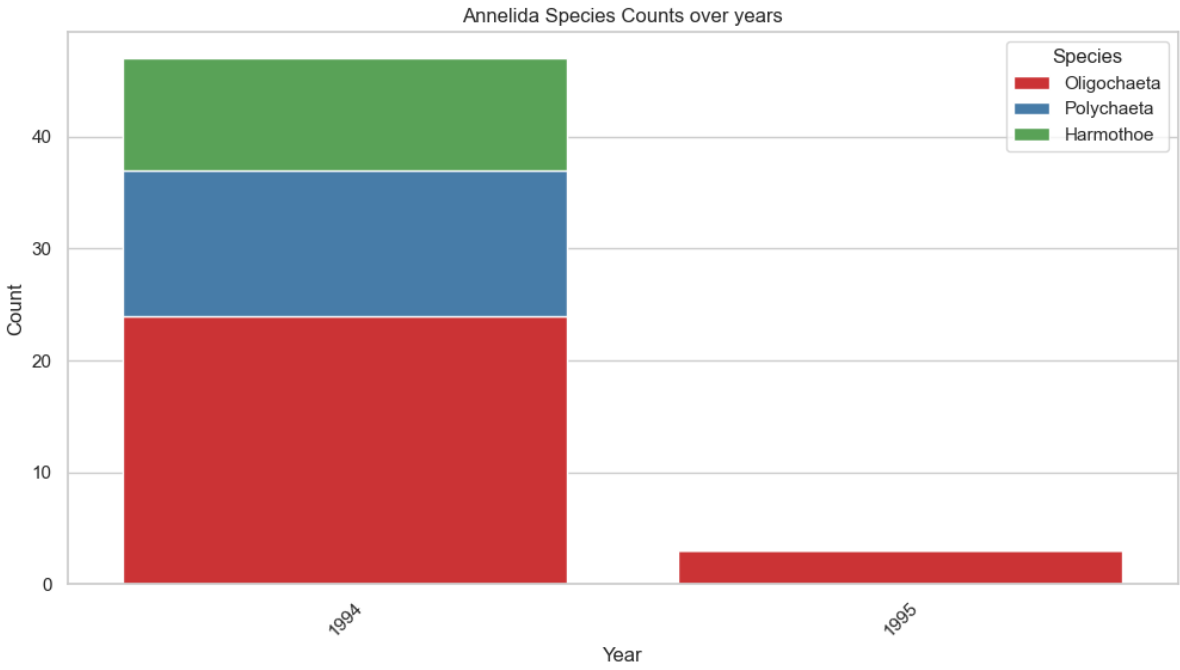
```
#Annelida

# Filter the DataFrame for rows where 'Phylum' is 'Annelida'
annelida_data = df[df['phylum'] == 'Annelida']

# Count the occurrences of each unique scientific name within the 'Annelida
scientificname_counts = annelida_data['scientificname'].value_counts().reset
scientificname_counts.columns = ['Scientific Name', 'Count']

# Filter for scientific names with a count greater than or equal to 7
scientificname_counts = scientificname_counts[scientificname_counts['Count']


scientificname_counts = scientificname_counts.sort_values(by='Count', ascenc

plt.figure(figsize=(12, 6))
sns.barplot(x='Scientific Name', y='Count', data=scientificname_counts, pale
plt.xlabel('Scientific Name')
plt.ylabel('Count')
plt.title('Species within Annelida')
plt.xticks(rotation=0)
plt.show()
```



In [239…

```
#Annelida count over years
```

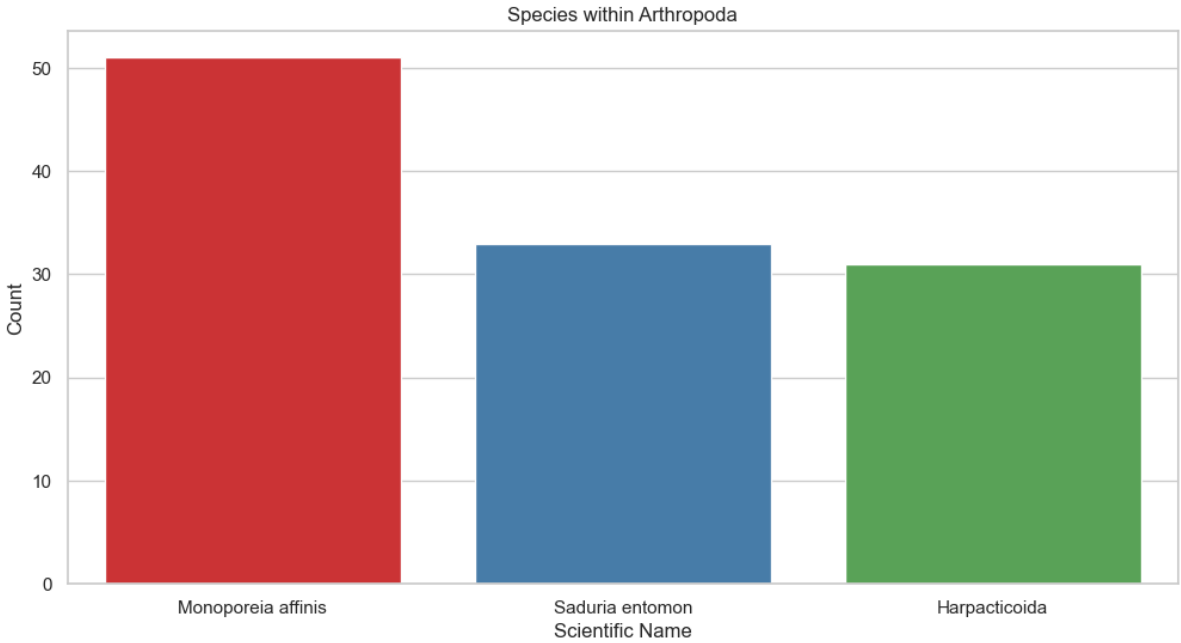
```
years_to_analyze = [1994, 1995]

species_to_plot = ['Oligochaeta', 'Polychaeta', 'Harmothoe']


filtered_data = df[df['yearcollected'].isin(years_to_analyze) & df['scientif

# Group the data by species and year, and count the occurrences
species_counts = filtered_data.groupby(['yearcollected', 'scientificname']).


plt.figure(figsize=(12, 6))
sns.barplot(x=species_counts.index, y=species_counts.sum(axis=1), color='lig
bottom = 0

for species in species_to_plot:
    sns.barplot(x=species_counts.index, y=species_counts[species], color=sns
    bottom += species_counts[species]

plt.xlabel('Year')
plt.ylabel('Count')
plt.title('Annelida Species Counts over years')
plt.xticks(rotation=45)
plt.legend(title='Species')
plt.savefig("species_counts_plot_Annelida_years_Baltic_sea.pdf", format="pdf
plt.show()
```



In [204…

```
#Anthropoda Count

# Filter the DataFrame for rows where 'Phylum' is 'Arthropoda'
arthropoda_data = df[df['phylum'] == 'Arthropoda']

# Count the occurrences of each unique scientific name within the 'Arthropoc
scientificname_counts = arthropoda_data['scientificname'].value_counts().res
scientificname_counts.columns = ['Scientific Name', 'Count']

# Filter for scientific names with a count greater than or equal to 30
scientificname_counts = scientificname_counts[scientificname_counts['Count']

scientificname_counts = scientificname_counts.sort_values(by='Count', ascenc
```

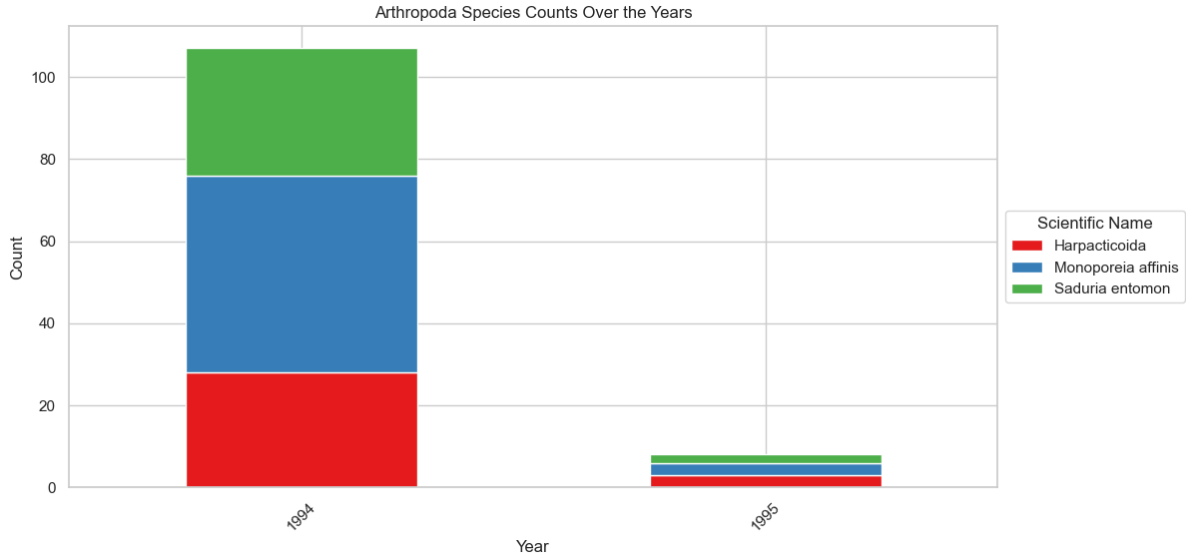
```
plt.figure(figsize=(12, 6))
sns.barplot(x='Scientific Name', y='Count', data=scientificname_counts, pale
plt.xlabel('Scientific Name')
plt.ylabel('Count')
plt.title('Species within Arthropoda')
plt.xticks(rotation=0)
plt.show()
```



In [238…

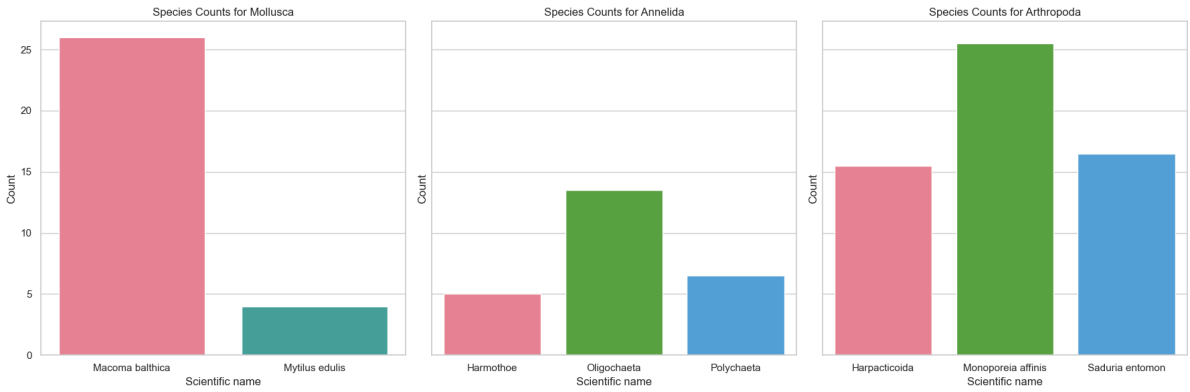
```
#Anthropoda over years


species_to_plot = ['Harpacticoida', 'Saduria entomon', 'Monoporeia affinis']

# Filter the DataFrame for rows where 'Phylum' is 'Arthropoda' and the speci
filtered_data = df[(df['phylum'] == 'Arthropoda') & (df['scientificname'].is

# Group the data by 'yearcollected' and 'scientificname' and count the occur
species_counts = filtered_data.groupby(['yearcollected', 'scientificname']).


plt.figure(figsize=(12, 6))
sns.set_palette("Set1")
ax = species_counts.plot(kind='bar', stacked=True, figsize=(12, 6))
plt.xlabel('Year')
plt.ylabel('Count')
plt.title('Arthropoda Species Counts Over the Years')
plt.xticks(rotation=45)
ax.legend(title='Scientific Name', loc='center left', bbox_to_anchor=(1, 0.5
plt.savefig("species_counts_over_years_Baltic_sea.pdf", format="pdf")
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```



In [271…

```
#All the phyla under one plot



# Defining specific species for each phylum
mollusca_species = ['Macoma balthica', 'Mytilus edulis']
annelida_species = ['Oligochaeta', 'Polychaeta', 'Harmothoe']
arthropoda_species = ['Monoporeia affinis', 'Saduria entomon', 'Harpacticoid

# Create subplots for each phylum
fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharey=True)

# Filter the DataFrame for each phylum and specific species
for i, (phylum, species) in enumerate(zip(['Mollusca', 'Annelida', 'Arthropo
    phylum_data = df[(df['phylum'] == phylum) & (df['scientificname'].isin(s

    species_counts = phylum_data.groupby(['yearcollected', 'scientificname']

    ax = sns.barplot(data=species_counts, ci=None, palette="husl", dodge=Fal
    ax.set_xlabel('Scientific name')
    ax.set_ylabel('Count')
    ax.set_title(f'Species Counts for {phylum}')
    ax.set_xticklabels(ax.get_xticklabels(), rotation=0)


plt.tight_layout()
plt.savefig("species_counts_plot_Baltic_sea.pdf", format="pdf")
plt.show()
```



In [249…

```
# shannon diversity index
import math
```

```
# Counts of the specified species
species_counts = {
    'Monoporeia affinis': 51,
    'Saduria entomon': 33,
    'Harpacticoida': 31,
    'Oligochaeta': 27,
    'Macoma balthica': 26,
    'Polychaeta': 13,
    'Harmothoe': 10,
    'Mytilus edulis': 4
}

# Calculate the total abundance in the sample
total_abundance = sum(species_counts.values())

# Calculate the relative abundance (Pi) for each species
relative_abundance = {species: count / total_abundance for species, count in

# Calculate the natural logarithm (ln) of each Pi value
ln_pi_values = {species: math.log(pi) for species, pi in relative_abundance.

# Calculate Pi * ln(Pi) for each species
pi_ln_values = {species: pi * ln_pi for species, pi, ln_pi in zip(relative_a

# Calculate the Shannon Diversity Index (H) for this sample
shannon_index = -sum(pi_ln_values.values())

# Print the Shannon Diversity Index
print(f"Shannon Diversity Index (H): {shannon_index:.3f}")
```

```
Shannon Diversity Index (H): 1.899
```

In [ ]:

In [272…

```
season_mapping = {"spring": 1, "summer": 0}

# Apply the mapping to the 'seasoncollected' column
df['seasoncollected'] = df['seasoncollected'].map(season_mapping)

print(season_mapping)
```

```
{'spring': 1, 'summer': 0}
```

In [273…

```
import pandas as pd

df_train = pd.read_csv('Baltic.csv')

# Select the columns you want to save
selected_columns = df_train[['seasoncollected', 'phylum', 'yearcollected']]


output_csv_file = 'baltic_model_train.csv'

# Save the selected columns to the CSV file
selected_columns.to_csv(output_csv_file, index=False)
```

In [274…

```
df_m = pd.read_csv('baltic_model_train.csv')
df_m.head(3)

print(df_m.columns)
```

```
Index(['seasoncollected', 'phylum', 'yearcollected'], dtype='object')
```

In [275…

```python
# Define the mapping of phylum names to binary values
phylum_mapping = {'Annelida': 0, 'Anthropoda': 1, 'Mollusca': 2}

# Map the 'phylum' column to binary values and create a new 'phylum_binary'
df_m['phylum_binary'] = df_m['phylum'].map(phylum_mapping)


print(df_m.head())
```

```
  seasoncollected      phylum  yearcollected  phylum_binary
0          spring    Annelida           1994            0.0
1          spring    Mollusca           1994            2.0
2          spring  Ochrophyta           1994            NaN
3          spring    Nemertea           1994            NaN
4          spring  Arthropoda           1994            NaN
```

In [276…

```python
# Drop rows with NaN values in the 'phylum_binary'
df_m = df_m.dropna(subset=['phylum_binary'])

# Map 'seasoncollected' values
season_mapping = {'spring': 0, 'summer': 1}
df_m['SC'] = df_m['seasoncollected'].map(season_mapping)

# Convert 'phylum_binary' column to integers
df_m['phylum_binary'] = df_m['phylum_binary'].astype(int)


print(df_m.head())
```

```
   seasoncollected    phylum  yearcollected  phylum_binary  SC
0           spring  Annelida           1994              0   0
1           spring  Mollusca           1994              2   0
20          spring  Annelida           1994              0   0
21          spring  Mollusca           1994              2   0
22          spring  Annelida           1994              0   0
```

In [277…

```python
selected_phyla = ['Mollusca', 'Annelida', 'Anthropoda']
df_m = df_m[df_m['phylum'].isin(selected_phyla)]
df_m.drop(columns=['seasoncollected', 'phylum'], inplace=True)
df_m.head()
```

Out[277]:

	yearcollected	phylum_binary	SC
0	1994	0	0
1	1994	2	0
20	1994	0	0
21	1994	2	0
22	1994	0	0

In [290…

```python
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split

# Separate the target variable from the features
X = df_m.drop(columns=['yearcollected'])
y = df_m['yearcollected']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

# Create and train the SVM model
```

```
clf = SVC(kernel='linear', C=1.0)
clf.fit(X_train, y_train)

# Make predictions on the test set
y_pred = clf.predict(X_test)

# Evaluate the model's performance
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9523809523809523
```

In [288…

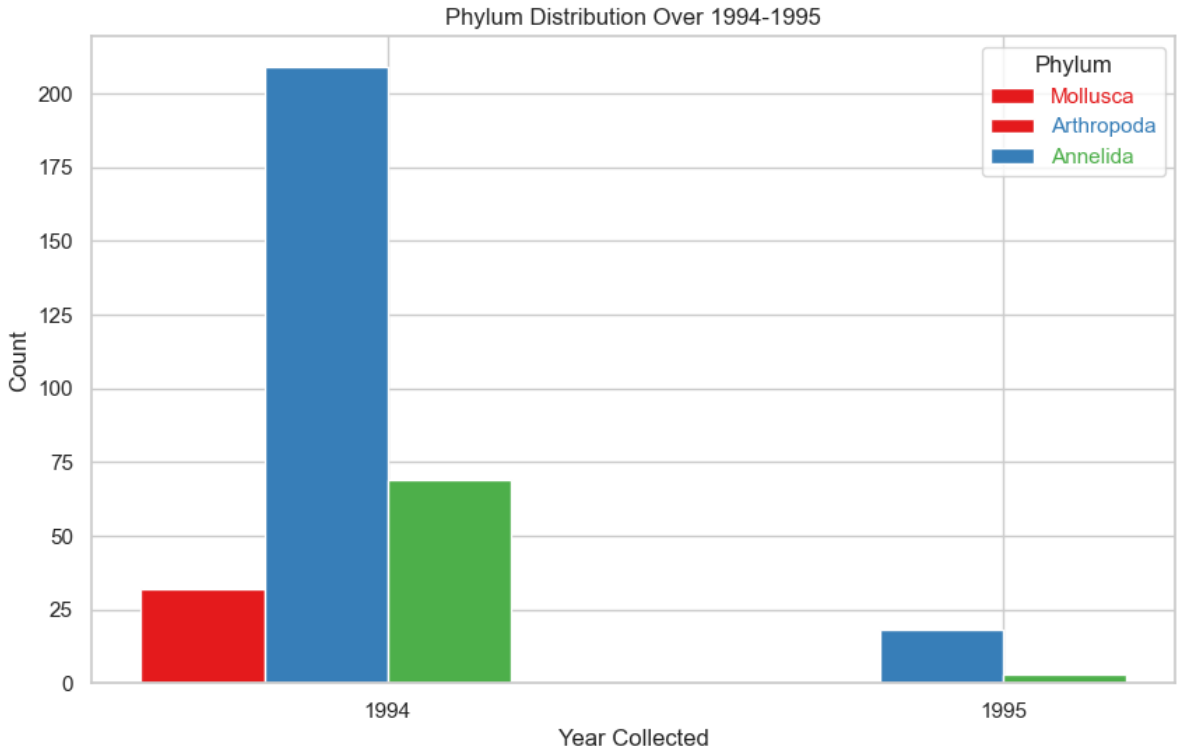
```
# Calculate the Pearson correlation coefficient and the p-value
correlation, p_value = pearsonr(df_m['phylum_binary'], df_m['yearcollected']

print(f"Pearson Correlation: {correlation:.2f}")
print(f"P-Value: {p_value:.2f}")
```

```
Pearson Correlation: -0.11
P-Value: 0.25
```

In [279…

```
import pandas as pd

tkc = pd.read_csv('Baltic_revised.csv')

# Filter the dataset for 'Phylum' values of 'Mollusca,' 'Annelida,' and 'Art
selected_phyla = ['Mollusca', 'Annelida', 'Arthropoda']
filtered_df =tkc[tkc['phylum'].isin(selected_phyla)]


selected_years = [1994, 1995]
filtered_df = filtered_df[filtered_df['yearcollected'].isin(selected_years)]
```

In [280…

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt



selected_phyla = ['Mollusca', 'Arthropoda', 'Annelida']

# Group the data by 'Year Collected' and 'Phylum' and calculate the count
grouped_data = filtered_df.groupby(['yearcollected', 'phylum']).size().unsta

# Define the years as categories
years = grouped_data.index.astype(str)

# Define a custom color palette for each phylum
colors = sns.color_palette("Set1", len(selected_phyla))


plt.figure(figsize=(10, 6))
bar_width = 0.2


bars = []
legend_labels = []

for i, phylum in enumerate(selected_phyla):
    count = grouped_data[phylum]
    x_positions = np.arange(len(years)) + i * bar_width
    bars += plt.bar(x_positions, count, width=bar_width, label=phylum, color
    legend_labels.append(phylum)
```

```
plt.title('Phylum Distribution Over 1994-1995')
plt.xlabel('Year Collected')
plt.ylabel('Count')
plt.xticks(np.arange(len(years)) + (bar_width * len(selected_phyla)) / 2, ye


legend = plt.legend(bars, legend_labels, title='Phylum')


for i, text in enumerate(legend.get_texts()):
    text.set_color(colors[i])

plt.savefig("phylum_over_years_Baltic.pdf", format="pdf")
plt.show()
```